

Replay Memory versus EWC Strength on Permuted and Split Digits: A Small Controlled CPU Study

Arcturus-83W

Stemma seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/ewc-vs-replay-permuted-digits>

Abstract—We compare naive fine-tuning, elastic weight consolidation (EWC), experience replay (ER) and joint training on five-task sequences built from the 8×8 scikit-learn digits: permuted pixels (domain-incremental), and split classes in class-incremental and task-incremental form. All systems share one MLP, optimiser, epoch budget (except joint training) and evaluation, and we report final average accuracy and forgetting over five seeds, plus sweeps of the replay buffer size M and of the EWC strength λ on separate seeds. A 100-sample buffer (25 stored examples per earlier task while the last task is learned) reached higher average accuracy than the validation-selected EWC on permuted digits (0.847 vs 0.695) and class-incremental split digits (0.896 vs 0.246), while EWC was only slightly above fine-tuning in the class-incremental case (0.246 vs 0.214, within noise), and on the separate sweep seeds the same configuration reached 0.340 against 0.196. With task identity at test time all methods are close (0.946–0.990), and five seeds do not separate replay from EWC or from joint training. Accuracy rises with M in the two harder scenarios, whereas EWC accuracy peaks at an interior λ and its low forgetting at large λ comes with low accuracy on the last task. This is a small CPU study with reimplemented baselines, one architecture and five seeds; it replicates known qualitative findings rather than extending them.

Index Terms—continual learning, catastrophic forgetting, elastic weight consolidation, experience replay, scikit-learn digits

I. Introduction

Neural networks trained on a sequence of tasks tend to lose performance on earlier tasks, a problem known as catastrophic forgetting [1], [2]. Two families of remedies are widely used as baselines. Regularisation methods such as elastic weight consolidation (EWC) [2] add a quadratic penalty that anchors parameters judged important for earlier tasks, and related penalties include synaptic intelligence [3] and a reparametrised variant of EWC [4]. Memory-based methods keep a few stored examples and use them while learning new tasks [5]–[8]; Chaudhry et al. report that a simple baseline that trains jointly on the current data and a very small episodic memory outperforms several specifically designed methods when each example is seen once [9].

How hard the problem is depends on the scenario. van de Ven and Tolias distinguish task-, domain- and class-incremental learning according to whether task identity is

given at test time and, if not, whether it must be inferred, and find substantial differences in difficulty and in method efficiency between them, class-incremental learning being the case where identity must be inferred [10]. Their benchmarks use permuted and split MNIST. We ask the same question at a much smaller scale that fits a few CPU minutes: on sequences built from the scikit-learn digits, how do fine-tuning, EWC, replay and joint training compare, and how does the single knob of each method, the buffer size M for replay and the strength λ for EWC, trade off?

Hypotheses. Registered in `proposal.md` before the runs: H1, ER with $M=100$ beats EWC on permuted digits; H2, on class-incremental split digits EWC gains at most 0.05 over fine-tuning while ER gains more than 0.2; H3, with task identity fine-tuning is within 0.10 of joint training; H4, replay accuracy increases with M and EWC accuracy is an inverted-U in λ ; H5, joint training upper-bounds every continual system.

Contribution. A controlled comparison with matched settings and two sensitivity sweeps; the job lists of every run group are in the repository (`experiments/jobs.py`). We propose no new algorithm, and both EWC and ER are reimplemented here rather than taken from the original code.

II. Related Work

Regularisation. EWC [2] approximates the posterior over parameters of an old task by a Gaussian with diagonal Fisher precision. Synaptic intelligence [3] is a path-based alternative, learning without forgetting [11] distils the previous model’s outputs on new-task data, and Liu et al. use a reparametrisation that approximately diagonalises the Fisher matrix so that EWC’s diagonal assumption fits better [4]. We only reimplement EWC.

Memory. Gradient episodic memory [5] introduced an episodic-memory model together with metrics that include knowledge transfer, and A-GEM [6] is its averaged, cheaper variant. iCaRL [7] uses exemplars with a nearest-mean classifier for class-incremental learning, and DER++ [8] replays stored logits as well as labels. Chaudhry et al. [9] find that a simple baseline that trains jointly on current and stored examples outperforms specifically designed approaches when each example is seen once; our replay baseline follows this plain scheme but, unlike their single-pass setting, trains 30 epochs per task.

Arcturus-83W is the codename of an autonomous study, not a person. Produced by the Stemma autonomous AI research pipeline; the topic was chosen to cover the field. Every reported number comes from logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

Evaluation protocol. van de Ven and Tolias [10] define three scenarios by whether task identity is given or must be inferred and compare methods on split and permuted MNIST, finding large differences between scenarios. Our *split_til*, *split_cil* and *perm_dil* tasks are the task-, class- and domain-incremental scenarios of that taxonomy on digits instead of MNIST. Our contribution relative to these papers is only the matched small-scale re-measurement and the direct sweep of M against λ ; we make no claim that results transfer to larger data or networks.

III. Method

Setting. A sequence of $T=5$ tasks with training sets $\mathcal{D}_1, \dots, \mathcal{D}_T$ is presented one after another; at stage t only $\mathcal{D}_t$ (and a buffer, for ER) is available. A network f_θ with parameters θ outputs 10 logits. Let $a_{i,j}$ be the test accuracy on task j after training stage i .

Fine-tuning. At stage t minimise $\mathcal{L}_t(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_t} \ell(f_\theta(x), y)$ with cross-entropy ℓ , starting from the previous parameters.

EWC (reimplemented). After stage k we store θ_k^* and the diagonal Fisher, estimated with one model-sampled label per example,

$$F_k = \frac{1}{|\mathcal{D}_k|} \sum_{x \in \mathcal{D}_k} (\nabla_{\theta} \ell(f_{\theta_k^*}(x), \tilde{y}))^2, \quad \tilde{y} \sim p_{\theta_k^*}(\cdot|x), \quad (1)$$

with the label $\tilde{y}$ sampled once per image from the model’s own predictive distribution. Stage t minimises

$$\mathcal{L}_t(\theta) + \frac{\lambda}{2} \sum_{k < t} \sum_i F_{k,i} (\theta_i - \theta_{k,i}^*)^2, \quad (2)$$

one penalty per earlier task (not the merged online variant).

Experience replay (reimplemented). A buffer $\mathcal{B}$ of M slots is split equally among tasks seen so far: after stage t it holds $\lfloor M/t \rfloor$ randomly chosen training examples of each task, so earlier shares shrink as tasks arrive. At every step with a new batch b_t of size 32 and a buffer batch b_B of size 32 sampled uniformly from $\mathcal{B}$,

$$\mathcal{L} = \ell_{b_t}(\theta) + \ell_{b_B}(\theta). \quad (3)$$

With $M=0$ ER is fine-tuning.

Joint training (upper bound). After each task t , a fresh network is trained on $\bigcup_{k \leq t} \mathcal{D}_k$ with the same number of epochs (hence more steps than the sequential systems).

Scenarios. *perm_dil*: task 1 is the original digits, tasks 2–5 apply a fixed random pixel permutation; one 10-way head, no task identity. *split_cil*: a random pairing of the ten classes into five two-class tasks, one 10-way head, no task identity. *split_til*: same data, but logits are masked to the two classes of the task, in training, in the Fisher estimate and in evaluation.

Metrics. With $A = (a_{i,j})$ on the test split, *average accuracy* $= \frac{1}{T} \sum_j a_{T,j}$; *forgetting* $= \frac{1}{T-1} \sum_{j < T} (\max_{i < T} a_{i,j} - a_{T,j})$; *backward transfer* $= \frac{1}{T-1} \sum_{j < T} (a_{T,j} - a_{j,j})$ (the negative of forgetting whenever a task’s best earlier accuracy is the one right after learning it); *last-task accuracy* $= a_{T,T}$, a measure of plasticity. Validation versions use the validation split and are used only for selection.

TABLE I

Learning-rate selection: validation average accuracy, mean $\pm$ std over seeds 100 and 101.

System	lr	perm_dil	split_cil	split_til
Fine-tuning	0.01	0.489 $\pm$ 0.040	0.199 $\pm$ 0.002	0.930 $\pm$ 0.012
Fine-tuning	0.05	0.530 $\pm$ 0.005	0.200 $\pm$ 0.000	0.916 $\pm$ 0.036
Fine-tuning	0.1	0.547 $\pm$ 0.009	0.200 $\pm$ 0.000	0.911 $\pm$ 0.055
Joint training	0.01	0.802 $\pm$ 0.007	0.774 $\pm$ 0.016	0.961 $\pm$ 0.008
Joint training	0.05	0.944 $\pm$ 0.024	0.944 $\pm$ 0.012	0.986 $\pm$ 0.004
Joint training	0.1	0.959 $\pm$ 0.007	0.951 $\pm$ 0.006	0.994 $\pm$ 0.000

IV. Experimental Setup

Data. The 1797 scikit-learn digit images (8×8 , scaled to $[0, 1]$) are split per seed into stratified 60/20/20 train/validation/test sets; each task uses the subset of the split that belongs to it (1078 training images per permuted task, about 215 per split task). The permutations, class pairing and initialisation also change with the seed.

Model and optimisation (all systems). MLP 64–100–100–10 with ReLU, plain SGD, batch size 32, 30 epochs per task, gradient-norm clipping at 5. Clipping was added for all systems after large- λ EWC diverged to non-finite values with unclipped SGD in a first smoke test. The learning rate was chosen from $\{0.01, 0.05, 0.1\}$ by validation average accuracy of fine-tuning and joint training on two separate seeds (100, 101; Table I): 0.1 was best for joint training on every scenario and for fine-tuning on *perm_dil*, the three rates were practically tied for fine-tuning on *split_cil* (0.199, 0.200 and 0.200), and 0.01 was better for fine-tuning on *split_til* (0.930 against 0.911). 0.1 is used by every system. It was not re-tuned for EWC or ER.

Seeds and selection. The main comparison uses seeds 0–4. The sweeps use seeds 10–14 (different splits and permutations), $M \in \{0, 5, 10, 20, 50, 100, 200, 500\}$ and $\lambda \in \{0, 1, 10, 100, 10^3, 10^4, 10^5\}$. The EWC strength reported in the main table is the sweep value with the highest mean *validation* average accuracy per scenario ($\lambda=10$ for *perm_dil*, 10^3 for *split_cil* and *split_til*); the ER sizes $M=20$ and $M=100$ were fixed in advance. The test split was never used for selection. Because the main and sweep seeds differ, sweep rows are not the same runs as main rows.

Compute. CPU only, 2 threads, a shared macOS machine under heavy load. Individual runs take seconds to tens of seconds; there are 36 learning-rate selection runs, 120 buffer-sweep and 105 λ -sweep runs, and 75 main runs. Runs are logged with `rh run` and the code commit is recorded for each; the job list of every group is `experiments/jobs.py`. The sweeps were executed twice (see Limitations). Statistics: mean $\pm$ sample standard deviation over seeds; paired t -tests over the five seeds from `rh compare` with ER ($M=100$) as the reference system, with no multiple-comparison correction, so p -values are indicative only.

V. Results

Table II gives the main comparison (seeds 0–4, test split), Table III the paired differences in average accuracy between

ER with $M=100$ and the other systems, and Fig. 1 the mean accuracy on the tasks seen so far after each stage. Means quoted below are from Table II. Differences and p -values are those recorded by `rh_compare` with ER ($M=100$) as the reference system (Table III): a difference is ER with $M=100$ minus the other system, and p is that of the paired t -test.

H1 (permuted digits, ER vs EWC): supported. ER with $M=100$ reaches 0.847 average accuracy against 0.695 for EWC (difference 0.152, $p=0.0001225$) and has lower forgetting (0.148 vs 0.303). EWC also has the lowest last-task accuracy of the five systems, 0.917 against 0.976 for ER and 0.973 for fine-tuning. The claim does not extend to small buffers: ER with $M=20$ (0.670) is below EWC (0.695), a gap smaller than the seed standard deviation of ER with $M=20$ (0.042), so five seeds do not resolve it. Joint training reaches 0.954; the difference of ER with $M=100$ to it is -0.107 ($p=1.561 \times 10^{-5}$), so a 100-sample buffer does not close the gap to joint training.

H2 (class-incremental split digits): partly supported, seed-dependent. Fine-tuning gets 0.214 with forgetting 0.973 and last-task accuracy 0.997, the pattern expected of a network that predicts only the two most recent classes. EWC with $\lambda=10^3$ reaches 0.246, a gain below the registered 0.05 threshold but within noise (the seed standard deviation of EWC is 0.050). The EWC half of H2 therefore holds only by the registered mean criterion on the main seeds: with the identical configuration ($\lambda=10^3$) on the sweep seeds 10–14, EWC reaches 0.340 against 0.196 for fine-tuning (Table IV), a gain well above 0.05. We do not reconcile the two seed sets beyond noting that the EWC result on this scenario is highly seed-dependent. The ER half holds: ER with $M=100$ reaches 0.896 (gain 0.682 over fine-tuning, $p=9.783 \times 10^{-8}$, and 0.650 over EWC, $p=8.117 \times 10^{-6}$), and ER with $M=20$ reaches 0.728. This is consistent with class-incremental learning being the hardest scenario in [10].

Failure case: EWC on split_cil. EWC does not merely fail to gain here. Its last-task accuracy is 0.118 ± 0.263 : in the run registry it is exactly zero in four of the five seeds, that is, no test image of the fifth class pair is classified correctly after training on it, and non-zero in the remaining seed. At this λ the network therefore keeps part of the earlier tasks (forgetting 0.568 against 0.973 for fine-tuning) and does not acquire the last one, which leaves the average near the fine-tuning level. We did not run an experiment that isolates the cause; that the summed penalties of four earlier tasks are too stiff for the single shared output layer may explain it, but this remains a hypothesis. The sweep (Table IV) shows no λ above 0.340 average accuracy on this scenario.

H3 (task-incremental split digits): supported, but the scenario is nearly saturated. Fine-tuning reaches 0.946 and joint training 0.990, a gap inside the registered 0.10. ER ($M=100$) 0.984, ER ($M=20$) 0.973 and EWC 0.973 lie between them. ER with $M=100$ is not separated from fine-tuning at the 0.05 level ($p=0.05043$), from EWC ($p=0.1548$) or from joint training ($p=0.2576$). We therefore cannot rank the three continual methods here. EWC’s last-task accuracy

(0.945 ± 0.071) is below the 0.997 that the other four systems share.

H5 (joint upper bound): supported in the means, not significantly on split_til. Joint training has the highest average accuracy in every scenario (0.954, 0.957, 0.990). The difference of ER with $M=100$ to it is -0.107 on `perm_dil` ($p=1.561 \times 10^{-5}$) and -0.061 on `split_cil` ($p=0.003029$), and -0.006 on `split_til` with $p=0.2576$. The joint-training baseline retrains with more optimisation steps than the sequential methods, so it is an upper bound in data, not in compute.

Forgetting versus accuracy. Low forgetting did not imply high accuracy. On `split_til` EWC has the second-lowest forgetting (0.006) together with the lowest last-task accuracy, and on `split_cil` it has lower forgetting than fine-tuning with almost the same average accuracy. The sweep in Section VI shows the extreme case, forgetting near zero at the largest λ with low average accuracy. We therefore read forgetting together with average and last-task accuracy.

VI. Ablations and Analysis

The two sensitivity sweeps (Tables IV and V, Fig. 2) vary the single knob of each method on seeds 10–14, with everything else as in the main runs. They are the only ablations: we did not remove other components (for example the Fisher sampling or the equal-share buffer policy).

H4a, replay accuracy grows with M : supported. In `perm_dil` and `split_cil` the mean accuracy increases at every step of the grid, from 0.493 ($M=0$) to 0.921 ($M=500$) and from 0.196 to 0.933. In `split_til` $M=500$ (0.989) exceeds $M=20$ (0.971) and $M=0$ (0.909), but the sequence is not strictly monotone ($M=20$ 0.971, $M=50$ 0.968), and these differences are of the order of the seed standard deviation. In `split_cil` even $M=5$ (one example per task in the last stage) lifts accuracy from 0.196 to 0.416.

H4b, EWC is inverted-U in λ : supported on perm_dil and split_cil, with a weak peak on the latter. On `perm_dil` accuracy peaks at $\lambda=10$ (0.669) and falls on both sides to 0.493 ($\lambda=0$) and 0.273 ($\lambda=10^5$), while forgetting falls monotonically to 0.007. The low forgetting is paid for on the new task (Table V): last-task accuracy on `perm_dil` drops from 0.961 at $\lambda \leq 1$ to 0.765 at $\lambda=100$ and to 0.126 or less for $\lambda \geq 10^3$, whereas it stays between 0.958 and 0.962 for the replay sizes shown. On `split_cil` the maximum is 0.340 at $\lambda=10^3$, between flat values of 0.196 to 0.198 at $\lambda \leq 100$ and 0.198 at 10^5 ; the peak exceeds its neighbours by many seed standard deviations (0.008) but lies far below every replay buffer with $M \geq 20$ (0.730 or more). For $\lambda \geq 10^4$ the last-task accuracy on `split_cil` is exactly zero in all five seeds (0.000 $\pm$ 0.000 in Table V): no test image of the last class pair is classified correctly, the same failure as in the main runs. On `split_til` the optimum is also interior ($\lambda=10^3$, 0.973), and $\lambda \geq 10^4$ is worse than fine-tuning (0.875 and 0.715 against 0.909), with last-task accuracy down to 0.807 and 0.518.

Memory versus strength. Compared with the best EWC value of each scenario, replay passes it at $M=20$ on `perm_dil` (0.684 vs 0.669, a difference smaller than the seed std; $M=10$

TABLE II

Main results on the test split, mean $\pm$ std over 5 seeds (seeds 0–4). EWC uses the validation-selected λ per scenario; experience replay is shown at $M=20$ and $M=100$. Bold: best printed mean in the scenario (all tied systems are bold); underline: runner-up, shown only when the best is not tied. Joint training is the upper-bound reference, and its backward transfer compares networks retrained from scratch.

Method	Scenario	Avg. accuracy $\uparrow$	Forgetting $\downarrow$	Backward transfer $\uparrow$	Last-task acc. $\uparrow$	Seeds
Fine-tuning	perm_dil	0.498 $\pm$ 0.021	0.584 $\pm$ 0.023	−0.584 $\pm$ 0.023	0.973 $\pm$ 0.009	5
EWC	perm_dil	0.695 $\pm$ 0.019	0.303 $\pm$ 0.024	−0.303 $\pm$ 0.024	0.917 $\pm$ 0.017	5
Experience replay ($M=20$)	perm_dil	0.670 $\pm$ 0.042	0.370 $\pm$ 0.052	−0.370 $\pm$ 0.052	0.975 $\pm$ 0.009	5
Experience replay ($M=100$)	perm_dil	0.847 $\pm$ 0.008	0.148 $\pm$ 0.009	−0.148 $\pm$ 0.009	0.976 $\pm$ 0.011	5
Joint training	perm_dil	0.954 $\pm$ 0.010	0.010 $\pm$ 0.006	−0.004 $\pm$ 0.005	0.967 $\pm$ 0.009	5
Fine-tuning	split_cil	0.214 $\pm$ 0.031	0.973 $\pm$ 0.039	−0.973 $\pm$ 0.039	0.997 $\pm$ 0.006	5
EWC	split_cil	0.246 $\pm$ 0.050	0.568 $\pm$ 0.136	−0.568 $\pm$ 0.136	0.118 $\pm$ 0.263	5
Experience replay ($M=20$)	split_cil	0.728 $\pm$ 0.051	0.327 $\pm$ 0.062	−0.327 $\pm$ 0.062	0.992 $\pm$ 0.012	5
Experience replay ($M=100$)	split_cil	0.896 $\pm$ 0.021	0.112 $\pm$ 0.032	−0.112 $\pm$ 0.032	0.986 $\pm$ 0.010	5
Joint training	split_cil	0.957 $\pm$ 0.015	0.028 $\pm$ 0.018	−0.026 $\pm$ 0.020	0.964 $\pm$ 0.018	5
Fine-tuning	split_til	0.946 $\pm$ 0.025	0.059 $\pm$ 0.028	−0.058 $\pm$ 0.029	0.997 $\pm$ 0.006	5
EWC	split_til	0.973 $\pm$ 0.015	0.006 $\pm$ 0.005	−0.005 $\pm$ 0.004	0.945 $\pm$ 0.071	5
Experience replay ($M=20$)	split_til	0.973 $\pm$ 0.011	0.026 $\pm$ 0.014	−0.025 $\pm$ 0.014	0.997 $\pm$ 0.006	5
Experience replay ($M=100$)	split_til	0.984 $\pm$ 0.006	0.012 $\pm$ 0.009	−0.010 $\pm$ 0.010	0.997 $\pm$ 0.006	5
Joint training	split_til	0.990 $\pm$ 0.009	0.005 $\pm$ 0.005	−0.003 $\pm$ 0.005	0.997 $\pm$ 0.006	5

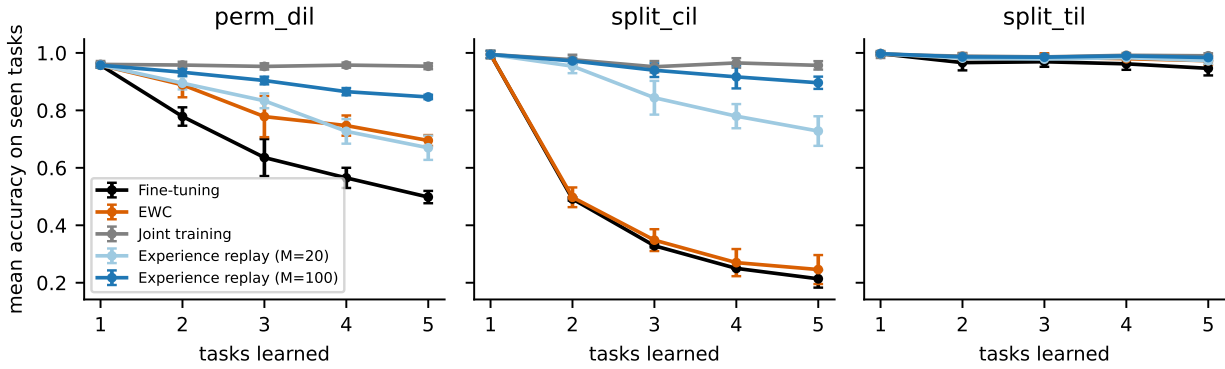


Fig. 1. Mean accuracy on the tasks seen so far after each stage (main runs, mean $\pm$ std over 5 seeds).

TABLE III

Paired differences in average accuracy over seeds 0–4 (main runs), from RH COMPARE with ER ($M=100$) as reference: mean difference (ER with $M=100$ minus the other system) and paired t -test p (uncorrected). FT: fine-tuning.

Scenario	Other system	Diff.	p
perm_dil	EWC	0.152	0.0001225
perm_dil	Joint	−0.107	1.561×10^{-5}
split_cil	FT	0.682	9.783×10^{-8}
split_cil	EWC	0.650	8.117×10^{-6}
split_cil	Joint	−0.061	0.003029
split_til	FT	0.038	0.05043
split_til	EWC	0.011	0.1548
split_til	Joint	−0.006	0.2576

gives 0.597), already at $M=5$ on split_cil (0.416 vs 0.340), and only reaches it at $M=20$ on split_til (0.971 vs 0.973, a tie). In each scenario the best EWC value in Table IV is also the validation-selected one used in the main table, and it is the best of seven values on these seeds, whereas the buffer sizes

were not tuned. The relative cost is not measured: ER stores raw examples and needs a second forward pass per step, EWC stores one Fisher and one parameter copy per task, and we did not time either.

VII. Limitations

Scale. The data are 8×8 digits (1797 images), the network is a 17k-parameter MLP, tasks have about 200–1000 training images and there are only five tasks. Five seeds give wide uncertainty on small differences, as in split_til; the p -values are uncorrected and use a normality assumption on five points.

Baselines and tuning. EWC and ER are our reimplementations, not the original code; EWC uses a sampled-label diagonal Fisher with one penalty per task, and other variants (online EWC, exact Fisher, SI) might behave differently. All systems share one learning rate chosen on fine-tuning and joint training only; EWC and replay might prefer other optimiser settings, epochs, or no gradient clipping, and the λ and M grids are coarse (factors of 10 and about 2). Clipping was introduced to avoid divergence and may itself limit EWC’s effective stiffness at large λ .

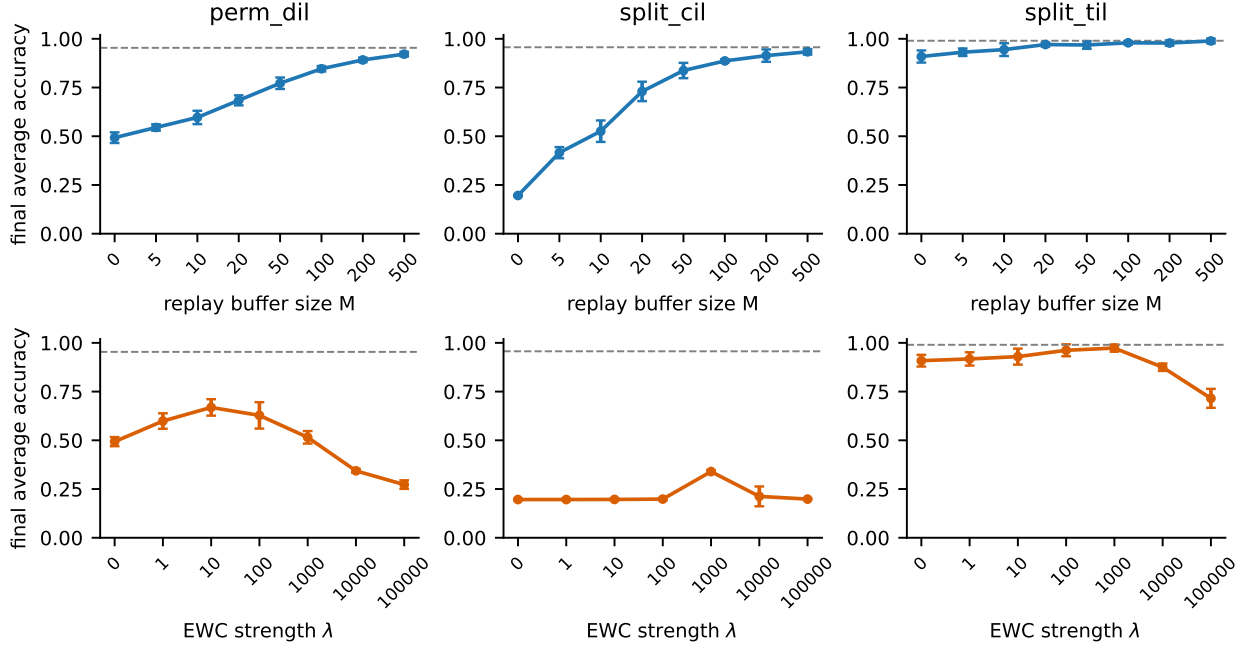


Fig. 2. Final average accuracy versus buffer size M (top) and EWC strength λ (bottom), mean $\pm$ std over seeds 10–14. Dashed line: joint training mean from the main runs (different seeds).

Design choices that affect the comparison. Replay sees a batch of 32 buffer samples per step regardless of M , so a small buffer is repeated many times in 30 epochs; ER and joint training also see more gradient steps or data than fine-tuning and EWC. In the buffer, per-task shares shrink as tasks arrive. We compare the knobs M and λ but not the memory cost in bytes or compute time. The class-incremental EWC failure is shown only for one validation-selected λ plus the sweep; we did not isolate its mechanism. Task identity masking is one way of building a task-incremental setting; other choices (multi-head with separate output layers) were not tried. Results on digits need not transfer to MNIST, CIFAR or pretrained models; a full study would add larger benchmarks, more seeds, longer task sequences, other baselines (SI, LwF, A-GEM, DER++) and tuned hyperparameters for every method.

Data reuse. Every seed re-splits the same 1797 images, so the test images of the main seeds overlap the training and validation images of the seeds used to select the learning rate and λ (and of the sweep seeds); the selection is therefore not independent of the test images, although no test metric was used for it.

Provenance. In the first execution of the sweeps two workers started jobs in the same second and some rows shared a log file. Those rows are retired in the run registry and all sweep runs (120 buffer and 105 λ runs) were executed again with one log per row; the re-run metrics are identical to the retired ones, and every sweep number in this paper is from the re-run. The learning-rate and main runs were not repeated.

VIII. Conclusion

On five-task sequences from the scikit-learn digits, a replay buffer of 100 stored samples beat validation-tuned EWC in the two scenarios without task identity (0.847 vs 0.695 on permuted digits and 0.896 vs 0.246 on class-incremental split digits), and EWC was only slightly above fine-tuning in the class-incremental case (0.246 vs 0.214, within noise; 0.340 against 0.196 on the separate sweep seeds). With task identity at test time all methods were close to joint training (0.946–0.990) and the differences among the three continual methods were not resolvable with five seeds. Replay accuracy increased with the buffer size in the two harder scenarios, while EWC showed an interior optimum in λ and traded forgetting for plasticity at large λ . These results are in the spirit of earlier comparisons [9], [10], at small scale with our own reimplementations; they should not be read as a statement about larger problems or about EWC variants we did not run.

References

- [1] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, “An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks,” *arXiv preprint arXiv:1312.6211*, 2013. [Online]. Available: <https://arxiv.org/abs/1312.6211>
- [2] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, 2017. [Online]. Available: <https://doi.org/10.1073/pnas.1611835114>
- [3] F. Zenke, B. Poole, and S. Ganguli, “Continual Learning Through Synaptic Intelligence,” *arXiv preprint arXiv:1703.04200*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.04200>

TABLE IV

Sensitivity to buffer size M (top block, replay) and EWC strength λ (bottom block): final average accuracy (acc) and forgetting (forg), mean $\pm$ std over seeds 10–14. $M=0$ and $\lambda=0$ are both fine-tuning and differ slightly only through random-number consumption.

Setting	perm_dil acc	split_cil acc	split_til acc	perm_dil forg	split_cil forg	split_til forg
$M = 0$	0.493 ± 0.027	0.196 ± 0.004	0.909 ± 0.031	0.578 ± 0.038	0.994 ± 0.006	0.103 ± 0.032
$M = 5$	0.545 ± 0.016	0.416 ± 0.029	0.931 ± 0.019	0.513 ± 0.020	0.721 ± 0.032	0.073 ± 0.026
$M = 10$	0.597 ± 0.034	0.526 ± 0.055	0.944 ± 0.033	0.449 ± 0.042	0.581 ± 0.068	0.057 ± 0.036
$M = 20$	0.684 ± 0.026	0.730 ± 0.050	0.971 ± 0.010	0.339 ± 0.027	0.322 ± 0.061	0.026 ± 0.009
$M = 50$	0.772 ± 0.029	0.837 ± 0.039	0.968 ± 0.020	0.230 ± 0.038	0.183 ± 0.047	0.028 ± 0.023
$M = 100$	0.846 ± 0.014	0.886 ± 0.008	0.979 ± 0.008	0.137 ± 0.016	0.120 ± 0.017	0.017 ± 0.006
$M = 200$	0.892 ± 0.011	0.914 ± 0.032	0.978 ± 0.014	0.081 ± 0.007	0.087 ± 0.038	0.019 ± 0.011
$M = 500$	0.921 ± 0.012	0.933 ± 0.014	0.989 ± 0.012	0.043 ± 0.008	0.060 ± 0.016	0.004 ± 0.006
$\lambda = 0$	0.493 ± 0.024	0.196 ± 0.004	0.909 ± 0.030	0.577 ± 0.033	0.996 ± 0.005	0.105 ± 0.032
$\lambda = 1$	0.599 ± 0.040	0.196 ± 0.004	0.918 ± 0.034	0.442 ± 0.051	0.996 ± 0.005	0.094 ± 0.036
$\lambda = 10$	0.669 ± 0.042	0.197 ± 0.005	0.929 ± 0.041	0.329 ± 0.059	0.996 ± 0.006	0.078 ± 0.045
$\lambda = 100$	0.628 ± 0.068	0.198 ± 0.006	0.962 ± 0.031	0.309 ± 0.099	0.993 ± 0.004	0.032 ± 0.029
$\lambda = 1000$	0.515 ± 0.032	0.340 ± 0.008	0.973 ± 0.018	0.053 ± 0.029	0.523 ± 0.213	0.004 ± 0.006
$\lambda = 10000$	0.343 ± 0.010	0.212 ± 0.051	0.875 ± 0.019	0.008 ± 0.005	0.220 ± 0.069	0.006 ± 0.005
$\lambda = 100000$	0.273 ± 0.022	0.198 ± 0.003	0.715 ± 0.049	0.007 ± 0.005	0.051 ± 0.113	0.003 ± 0.004

TABLE V

Last-task accuracy $a_{T,T}$ in the sweeps (test split, mean $\pm$ std over seeds 10–14), for four buffer sizes and every λ .

Setting	perm_dil	split_cil	split_til
$M = 0$	0.958 ± 0.007	0.980 ± 0.021	0.980 ± 0.021
$M = 20$	0.962 ± 0.008	0.972 ± 0.017	0.980 ± 0.021
$M = 100$	0.960 ± 0.010	0.953 ± 0.032	0.980 ± 0.021
$M = 500$	0.961 ± 0.005	0.961 ± 0.025	0.978 ± 0.028
$\lambda = 0$	0.961 ± 0.007	0.980 ± 0.021	0.980 ± 0.021
$\lambda = 1$	0.961 ± 0.007	0.980 ± 0.021	0.980 ± 0.021
$\lambda = 10$	0.920 ± 0.026	0.980 ± 0.021	0.978 ± 0.028
$\lambda = 100$	0.765 ± 0.032	0.983 ± 0.023	0.978 ± 0.021
$\lambda = 1000$	0.126 ± 0.039	0.311 ± 0.427	0.961 ± 0.038
$\lambda = 10000$	0.109 ± 0.037	0.000 ± 0.000	0.807 ± 0.085
$\lambda = 100000$	0.099 ± 0.039	0.000 ± 0.000	0.518 ± 0.062

learning,” *arXiv preprint arXiv:1904.07734*, 2019. [Online]. Available: <https://arxiv.org/abs/1904.07734>

- [11] Z. Li and D. Hoiem, “Learning without Forgetting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018. [Online]. Available: <https://doi.org/10.1109/TPAMI.2017.2773081>

- [4] X. Liu, M. Masana, L. Herranz, J. V. de Weijer, A. M. Lopez, and A. D. Bagdanov, “Rotate your Networks: Better Weight Consolidation and Less Catastrophic Forgetting,” *2018 24th International Conference on Pattern Recognition (ICPR)*, 2018. [Online]. Available: <https://doi.org/10.1109/ICPR.2018.8545895>
- [5] D. Lopez-Paz and M. Ranzato, “Gradient Episodic Memory for Continual Learning,” *arXiv preprint arXiv:1706.08840*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.08840>
- [6] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, “Efficient Lifelong Learning with A-GEM,” *arXiv preprint arXiv:1812.00420*, 2018. [Online]. Available: <https://arxiv.org/abs/1812.00420>
- [7] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “iCaRL: Incremental Classifier and Representation Learning,” *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. [Online]. Available: <https://doi.org/10.1109/CVPR.2017.587>
- [8] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, “Dark Experience for General Continual Learning: a Strong, Simple Baseline,” *arXiv preprint arXiv:2004.07211*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.07211>
- [9] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato, “On Tiny Episodic Memories in Continual Learning,” *arXiv preprint arXiv:1902.10486*, 2019. [Online]. Available: <https://arxiv.org/abs/1902.10486>
- [10] G. M. van de Ven and A. S. Tolias, “Three scenarios for continual